CSE 423 : Computer Graphics

VAULT RUNNER

A 3D Stealth Heist Game in OpenGL

Project Specification Document

Group Members

#	Full Name	Student ID	Primary Role
1	Tasbir Ahmed	22201028	
2	Aryan Rafsanjany	22299025	
3	Fatin Rahman Taseen	22299002	

Field	Detail
Section	15
Faculty	SRU,ZLNM
Project Type	3D Game (3D OpenGL)
Total Features	18 game features

1. Project Overview

Vault Runner is a single player 3D stealth game built entirely in PyOpenGL using the GLUT toolkit. The player controls an infiltrator who must move through a guarded vault facility, collect valuables scattered across the floor, and reach the extraction door before a countdown timer expires, all while avoiding detection by patrolling guards, rotating security cameras, and timed laser tripwires.

The central design idea is that the player is never given a weapon. Where Lab Assignment 3 rewarded aggressive play through shooting, Vault Runner inverts the loop entirely: the only tools available are movement, timing, positioning, and the deliberate management of noise. Every threat in the game must be read and avoided rather than eliminated. This inversion is what makes the project distinct from a straightforward extension of the lab work.

The technical core of the project is a four state guard artificial intelligence system driven by a geometric detection model. Each guard evaluates the player against a vision cone using a dot product test combined with a distance check, and transitions between behavioural states based on accumulated suspicion. This produces emergent gameplay: guards investigate where they last saw something, call for backup, and eventually return to patrol if the player breaks line of sight for long enough.

2. Gameplay Description

2.1 Objective

The player must collect loot worth at least the extraction threshold and then reach the exit door before the mission timer reaches zero. The exit door remains sealed and renders in red until the threshold is met, at which point it turns green and becomes usable. Collecting beyond the threshold is optional and increases the final score, which is the risk versus reward decision at the heart of each run.

2.2 Loss Conditions

A run ends in failure under any of three conditions. The player is caught when a guard in the chase state makes contact. The alarm level reaches its maximum of one hundred. Or the mission timer expires. On failure the player model collapses to the floor using a rotation transform, the screen text switches to a summary of the attempt, and the R key restarts the mission with a freshly generated layout.

2.3 The Core Loop

A typical thirty second window of play proceeds as follows. The player enters a room and immediately switches to the elevated third person camera to survey guard positions. They identify a guard whose patrol takes them away from a loot pedestal, crouch to halve their movement speed and noise output, and move along the wall while the guard is facing away. They collect the loot, notice a security camera sweeping toward their position, and step into a hiding zone until the sweep passes. They then move to the next room, misjudge the timing of a laser tripwire, trigger it, and must retreat to a hiding spot while a guard investigates the noise before resuming.

3. In-Game Feature Specification

The project implements eighteen distinct in-game features. Each is described below with its behaviour and its implementation approach. Section 9 maps these against the Lab Assignment 3 feature set to demonstrate that the three times requirement is satisfied.

1 Procedural Facility Layout

The vault floor is generated at runtime, never hardcoded. A grid of rooms is created by looping over cell indices derived from GRID_LENGTH, then interior walls are placed along cell boundaries with randomly positioned doorway gaps. Every restart produces a different layout, so the level cannot be memorised. Floor tiles alternate colour based on the parity of their grid indices, matching the technique demonstrated in the template.

2 Articulated Player Character

The infiltrator is assembled from primitives: a sphere for the head, cuboids for the torso and satchel, and cylinders for the limbs. The legs swing through a sine driven walk cycle whose amplitude scales with movement speed, so the character visibly slows and shortens its stride when crouching. On mission failure the entire model rotates ninety degrees about the Y axis to fall to the floor.

3 Guard Patrol System with Waypoints

Each guard owns an ordered list of waypoints generated alongside the level. The guard moves toward its current waypoint, and on arrival within a tolerance radius advances to the next, wrapping at the end to form a closed loop. Guards smoothly interpolate their facing angle toward their direction of travel rather than snapping, which makes their vision cone sweep readably as they turn corners.

4 Four State Guard AI

Every guard runs a finite state machine with the states PATROL, SUSPICIOUS, ALERT, and CHASE. PATROL follows the waypoint loop. SUSPICIOUS is entered when suspicion crosses a lower threshold; the guard stops and turns toward the stimulus. ALERT sends the guard to the last known player position to investigate, then waits before standing down. CHASE is entered on full detection and the guard pursues the player directly at increased speed. Each state has a distinct guard body colour so the player can read the AI state at a glance.

5 Vision Cone Detection

Detection is computed geometrically rather than approximated. The vector from guard to player is normalised and a dot product is taken against the guard's forward vector. The player is inside the cone when that dot product exceeds the cosine of the half angle and the distance is within the guard's sight range. The cone is drawn on the floor as a triangle fan of flat quads, tinted yellow while patrolling and red once alerted.

6 Graduated Suspicion Meter

Detection is not instantaneous. While the player is inside a vision cone, that guard's personal suspicion value rises at a rate inversely proportional to distance, so being spotted at close range escalates far faster than being glimpsed across a room. When the player leaves the cone, suspicion decays over time. This creates the central stealth tension of briefly crossing an exposed gap and accepting a partial fill of the meter.

7 Line of Sight Wall Occlusion

A guard cannot see through walls. Before a detection is confirmed, the segment from guard to player is tested against every interior wall using a segment intersection routine. If any wall blocks the segment, detection is rejected for that frame. This makes walls genuinely useful cover and turns the procedural layout into a tactical map rather than decoration.

8 Crouch Mode

Pressing SHIFT toggles crouch. The player model compresses vertically, movement speed halves, and the noise radius produced by movement drops to near zero. Crouching also reduces the effective distance at which guards accumulate suspicion, modelling a lower profile. The tradeoff against the mission timer is deliberate: stealth costs time.

9 Noise Propagation System

Movement emits noise into the world. Walking produces a moderate radius, sprinting produces a large one, and crouching produces almost none. Triggering a laser or colliding with an object emits a sharp one time burst. Any guard within the noise radius raises its suspicion and turns toward the origin point, independent of vision. The active noise radius is drawn as an expanding ring outline on the floor so the player can see exactly how far their movement carries.

10 Rotating Security Cameras

Wall mounted cameras sweep back and forth through a fixed arc. Each camera is drawn as a cuboid housing with a cylindrical lens, and projects its own narrower vision cone onto the floor. A camera that detects the player does not chase, but adds directly to the global alarm level and marks the player's position for nearby guards, effectively acting as a stationary sentry that converts position into information.

11 Timed Laser Tripwires

Laser grids guard corridors and high value rooms. Each emitter runs on an independent cycle with its own period and phase offset, drawn as thin elongated cuboids in bright red when armed and dimmed when inactive. Crossing an armed beam triggers an immediate alarm increase and a large noise burst. Because the cycles are independent, corridors present a genuine timing puzzle rather than a uniform on and off rhythm.

12 Hiding Zones

Marked zones such as ventilation alcoves and storage recesses are placed during generation. While the player stands inside one, suspicion accumulation is suppressed and existing suspicion decays at an accelerated rate. Zones are drawn as translucent floor markers with a distinct colour so they are identifiable at a distance, giving the player a reachable goal when a patrol goes wrong.

13 Global Alarm Level

A facility wide alarm value from zero to one hundred rises on every detection event and decays slowly during clean play. It is drawn as a bar in the heads up display. Crossing thresholds has escalating consequences: at forty guards patrol faster, at seventy their vision range extends, and at one hundred the mission fails outright. This ties the individual moments of a run into a single cumulative pressure.

F14 Tiered Loot and Collection

Loot items are drawn as pedestals topped with rotating primitives, colour coded by tier: bronze, silver, and gold, worth increasing point values. Higher tiers are placed by the generator inside areas with the densest

guard coverage. Collection occurs on proximity, plays a brief scale up and fade animation, and updates the carried total in the heads up display.

15 Conditional Extraction Door

The exit door remains locked until the carried loot value meets the extraction threshold. While locked it renders red with a visible barrier and displays the remaining requirement in the heads up display; once satisfied it turns green and the barrier is removed. Reaching it completes the mission and awards a time bonus proportional to the remaining seconds.

16 Mission Countdown Timer

Each run is bounded by a countdown driven by the same delta time value as all other systems. The remaining time is displayed in the heads up display and the text switches to a warning colour in the final thirty seconds. Expiry ends the run in failure regardless of loot carried, which is what prevents indefinitely patient play.

17 Dual Camera System

Right click toggles between a first person view positioned at the character's eye level and an elevated third person view that orbits the facility. The arrow keys control orbit angle and height in third person. The two views serve different purposes: third person for reading patrol patterns and planning, first person for the tension of executing a move without full information.

18 Tactical Overlay Mode

Pressing V toggles an overhead tactical overlay rendered through gluOrtho2D in the same push and pop pattern the template uses for text. It shows the facility outline, guard positions and facing, loot markers, and the exit, as a compact map in the screen corner. The overlay is limited by a rechargeable battery meter so it cannot be left on permanently, keeping it a tactical resource rather than a permanent solution.

4. Guards : AI State Machine

The guard artificial intelligence is the most substantial engineering component of the project and is specified here in full. Each guard instance maintains its own state, suspicion value, last known player position, and investigation timer.

State	Colour	Behaviour	Transitions Out
PATROL	Green	Follows the waypoint loop at base speed, facing interpolated toward direction of travel.	To SUSPICIOUS when suspicion exceeds 30.
SUSPICIOUS	Yellow	Halts and rotates to face the stimulus. The vision cone narrows slightly and the sight range extends.	To ALERT above 70. Back to PATROL below 15.
ALERT	Orange	Moves to the last known player position and searches, waiting there for a fixed interval.	To CHASE at 100. To PATROL when the search timer expires.
CHASE	Red	Pursues the player position directly at increased speed. Continuously raises the global alarm.	To ALERT when line of sight is lost for two seconds.

5. Control Scheme

All controls are bound exclusively through glutKeyboardFunc, glutSpecialFunc, and glutMouseFunc, the three input callbacks present in the template.

Input	Action	Notes
`W / S`	Move forward and backward	Discrete step per key event, scaled by crouch state
`A / D`	Rotate character left and right	Also rotates the first person view direction
`SHIFT`	Toggle crouch	Halves speed, suppresses noise, lowers profile
`SPACE`	Sprint burst	Temporary speed increase with a large noise radius
`E`	Collect loot	Active only when within pickup range of a pedestal
`V`	Toggle tactical overlay	Consumes the overlay battery while active
`P`	Pause and resume	Freezes all delta driven systems
`R`	Restart mission	Regenerates the layout and resets all counters
`Arrow Up / Down`	Camera height	Third person only

Input	Action	Notes
`Arrow Left / Right`	Camera orbit angle	Third person only
`Left Click`	Interact	Context sensitive: doors and terminals
`Right Click`	Toggle camera mode	Switches first person and third person

6. Rendering Architecture (Process)

Because depth testing is unavailable, the renderer implements the painter's algorithm explicitly. Each frame the display callback builds a list of drawable entries, where each entry carries a squared distance from the camera and a reference to its draw routine. The list is sorted in descending order of distance and traversed, so nearer objects overwrite farther ones in the colour buffer. The floor and walls are drawn first as a static base layer since they are always behind the dynamic objects.

Depth is further communicated through manual face shading. Each cuboid face is drawn with its base colour multiplied by a fixed brightness factor: top faces at full brightness, side faces at approximately seventy percent, and faces oriented away from the notional light at half. This is applied through glColor3f before each quad and produces a convincing sense of solidity without any lighting calls.

6.1 Primitive Budget

Entity	Primitives	Approximate Count per Frame
Floor tiles	GL_QUADS	144 tiles on a twelve by twelve grid
Walls and boundaries	GL_QUADS	40 to 60 segments depending on layout
Player	Sphere, cylinders, cuboids	9 primitives
Guards	Sphere, cylinders, cuboids	8 primitives each, 4 to 6 guards
Vision cones	GL_QUADS fan	12 quads per guard and camera
Loot and pedestals	Cuboids and spheres	2 primitives per item, 8 to 12 items
Lasers	Scaled cuboids	1 per emitter, 6 to 10 emitters
Overlay and HUD	gluOrtho2D quads and text	Approximately 30 elements

7. Data Model

All game state is held in module level dictionaries and lists, initialised through a single reset function that is called once at startup and again whenever the player restarts. Keeping initialisation and restart in one routine guarantees the two paths can never diverge as features are added.

Structure	Fields
`guards`	x, y, angle, state, suspicion, waypoints, wp_index, last_seen_x, last_seen_y, search_timer, speed, sight_range
`cameras`	x, y, base_angle, sweep_range, sweep_phase, detected
`lasers`	x1, y1, x2, y2, period, phase, armed
`loot`	x, y, tier, value, collected, spin_angle
`hide_zones`	x, y, radius
`walls`	x1, y1, x2, y2
`player`	x, y, angle, crouched, noise_radius, carried_value
`mission`	time_left, alarm_level, overlay_battery, state, score

8. Work Distribution

The project is divided into three workstreams with deliberately minimal coupling. Each member owns a set of features end to end, and the shared data structures listed in Section 8 form the contract between them.

Member	Workstream	Owned Features	Deliverable
Member 1	Rendering and Level Systems	F1, F2, F7, F10, F11, F14, F15	Procedural generator, painter's algorithm renderer, all static geometry and props
Member 2	Guard AI and Detection	F3, F4, F5, F6, F9, F13	State machine, detection mathematics, vision cone rendering, noise and alarm systems
Member 3	Player, HUD and Game Loop	F8, F12, F16, F17, F18	Player controller, camera system, heads up display, tactical overlay, timing and restart

9 Development Schedule

Phase	Focus	Exit Criteria
Week 1	Foundation	Procedural floor and walls render. The player moves and collides. Delta timing loop verified stable.
Week 1	Threat systems	Guards patrol waypoints. Vision cones draw and detect. State machine transitions correctly.
Week 1	Game systems	Loot, lasers, cameras, hiding zones, alarm, and timer are all functional. Win and loss conditions fire.
Week 2	Polish and integration	Tactical overlay complete. Face shading applied. Balance pass on patrol timing and thresholds.
Week 2	Verification	Full API audit against the template. Playtesting. Documentation and demonstration rehearsal.